

Lecture: 0100

Number Systems & Two's Complement

Binary · Hex · Signed Integers · Overflow

How the adders you just built actually represent numbers — and when they silently go wrong.

CSA222: Modern Computer Architecture

Where We Left Off

We built adders in L3 — but what do those bits actually mean?



Full adders & ripple chains

We can add two multi-bit binary numbers, carry and all (L3).



MUX, decoder, register

The block library that builds a datapath (L3).



But... what IS a number?

An adder just moves bits. Today: what those bit patterns represent.



...and negatives?

How does 8 bits store -5? And when does $127 + 1$ become -128?

An adder is 'dumb' — it flips bits by rules. The MEANING of those bits is a design decision. Today we make it.

Today's Roadmap

Four steps: represent, sign, compute, and catch the errors.

1

Number Systems

Binary, decimal, hex —
and fluent conversion
between all three.

2

Signed Integers

Sign-magnitude, 1's
complement, and the
winner: two's
complement.

3

Binary Arithmetic

Addition, and subtraction
done as addition of a
negative.

4

Overflow

When results silently wrap
around — and how
hardware detects it.

The punchline: two's complement lets ONE adder circuit do both addition and subtraction — no separate hardware.

Why Representation Matters

The same 8 bits can mean 255, or -1, depending on the rules you pick.

11111111

one byte, two readings

unsigned = 255

signed (2's comp) = -1

Same bits. The CPU only knows which you meant by which instruction you use.



It's a convention

Bits carry no inherent meaning — representation is a rulebook humans agreed on.



Bugs live here

The Ariane 5 rocket, Gangnam Style's view counter — real failures from bad number handling.



Hardware cares

The right representation lets one circuit add AND subtract — huge silicon savings.



You'll hit this in code

Every int overflow, every signed/unsigned cast bug traces back to today.

1. Number Systems

Binary is how the machine counts. Hex is how humans read binary without going insane.

Positional Notation

Every digit's value = the digit × the base raised to its position.

Decimal (base 10) — you already do this:

2	3	7
---	---	---

$$= 2 \times 100 + 3 \times 10 + 7 \times 1 = 237$$

$10^2=100$ $10^1=10$ $10^0=1$

Binary (base 2) — same idea, base 2:

1	0	1	1	0	1
---	---	---	---	---	---

$$= 32 + 8 + 4 + 1 = 45$$

32 16 8 4 2 1

The only difference between bases: which powers you use as place values. Base 2 → powers of 2. Base 16 → powers of 16.

Binary ↔ Decimal Conversion

Two directions, two methods — both worth having automatic.

BINARY → DECIMAL

Add up the place values where there's a 1.

1	0	1	0	1	0
32	16	8	4	2	1

$$32 + 8 + 2 = 42$$

Just read off the 1-columns and sum their weights.

DECIMAL → BINARY

Repeatedly divide by 2; read remainders bottom-up.

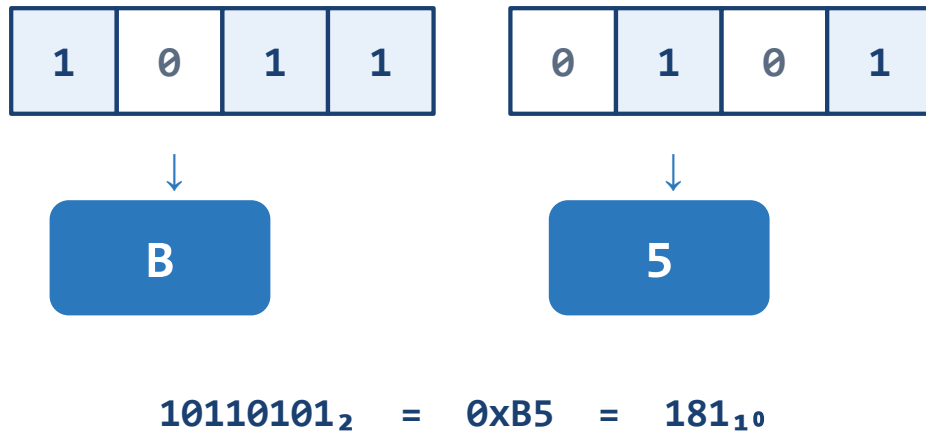
```
42 ÷ 2 = 21  r 0
21 ÷ 2 = 10  r 1
10 ÷ 2 = 5   r 0
5 ÷ 2 = 2    r 1
2 ÷ 2 = 1    r 0
1 ÷ 2 = 0    r 1
```

read up ↑ = 101010

Hexadecimal: Binary for Humans

One hex digit = exactly 4 bits. That's the whole reason it exists.

Group binary into nibbles of 4 — each becomes one hex digit:



HEX DIGITS BEYOND 9

Hex	Dec	Binary
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Two hex digits = one byte. That's why colors (#FF8800), memory addresses, and MAC addresses all use hex.

Worked Example: 0x2A

Convert one hex value to binary and decimal, step by step.

STEP 1 — Hex to binary (nibble each digit)

2

→ 0010

A

→ 1010

0x2A = 0010 1010₂

STEP 2 — Binary to decimal (sum the 1-weights)

0	0	1	0	1	0	1	0
128	64	32	16	8	4	2	1

32 + 8 + 2 = 42₁₀

ANSWER

0x2A

=

0010 1010₂

=

42₁₀

Powers of 2 — Memorise These

The multiplication table of computer architecture.

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
1	2	4	8	16	32	64	128	256

THE BIG MILESTONES

Power	Value	Name
2^{10}	1,024	1 KB
2^{20}	1,048,576	1 MB
2^{30}	~1 billion	1 GB
2^{40}	~1 trillion	1 TB

WHY IT PAYS OFF

Instantly know an n-bit range.

8 bits $\rightarrow 2^8 = 256$ values (0–255).

16 bits $\rightarrow 65,536$. 32 bits $\rightarrow$ ~4.3 billion.

This is why a 32-bit counter overflows at ~4.3 billion — the exact bug that broke YouTube's view counter.

2. Signed Integers

Binary is naturally unsigned. Representing negatives took three tries to get right.

Attempt 1: Sign-Magnitude

The obvious idea: use the top bit as a sign flag. Intuitive — but flawed.

Leftmost bit = sign (0 = +, 1 = -). Remaining 7 bits = magnitude.

0	0	0	1	0	0	1	0	= +18
---	---	---	---	---	---	---	---	-------

1	0	0	1	0	0	1	0	= -18
---	---	---	---	---	---	---	---	-------

Only the sign bit flips between +18 and -18.

PROBLEM

00000000 = +0 and 10000000 = -0. Two zeros! And addition needs special sign-checking logic — extra hardware.

VERDICT

✓ Easy for humans to read.

✗ Two representations of zero.

✗ Addition doesn't 'just work' — the hardware must inspect signs and decide whether to add or subtract.

Used today only for floating-point sign bits — never for integers.

Attempt 2: One's Complement

To negate, flip every bit. Closer — but zero is still doubled.

Negate by inverting ALL bits (NOT each one).

0	0	0	1	0	0	1	0
---	---	---	---	---	---	---	---

= +18

flip every bit ↓

1	1	1	0	1	1	0	1
---	---	---	---	---	---	---	---

= -18

STILL A PROBLEM

00000000 = +0 and 11111111 = -0. Two zeros again. Also needs an 'end-around carry' fix for addition.

PROGRESS

✓ Negation is trivial: just NOT.

✓ Addition nearly works...

✗ ...but needs the carry-out added back in ('end-around carry').

✗ STILL two zeros.

So close. One tiny tweak fixes everything →

Attempt 3: Two's Complement ✓

Flip all bits, then add 1. This one tweak fixes everything.

Negate: invert every bit, then add 1.

0	0	0	1	0	0	1	0	+18
invert ↓								
1	1	1	0	1	1	0	1	(1's comp)
add 1 ↓								
1	1	1	0	1	1	1	0	= -18

The trick: negation and addition use the SAME hardware. That's the entire payoff.

WHY IT WINS

✓ Exactly ONE zero (00000000).

✓ Ordinary binary addition just works — for positives AND negatives, no special cases.

✓ Subtraction = add the negative. ONE adder circuit does everything.

This is why every modern CPU uses two's complement for signed integers. Universal.

Reading a Two's Complement Number

The top bit has NEGATIVE weight. That's the whole secret.

The most-significant bit's place value is negative:

1	1	1	0	1	1	1	0
-128	64	32	16	8	4	2	1

$$-128 + 64 + 32 + 8 + 4 + 2 = -18$$

The -128 weight (red) is what makes the number negative; the rest add back positive value.

NEGATE = INVERT + 1 (works both ways)

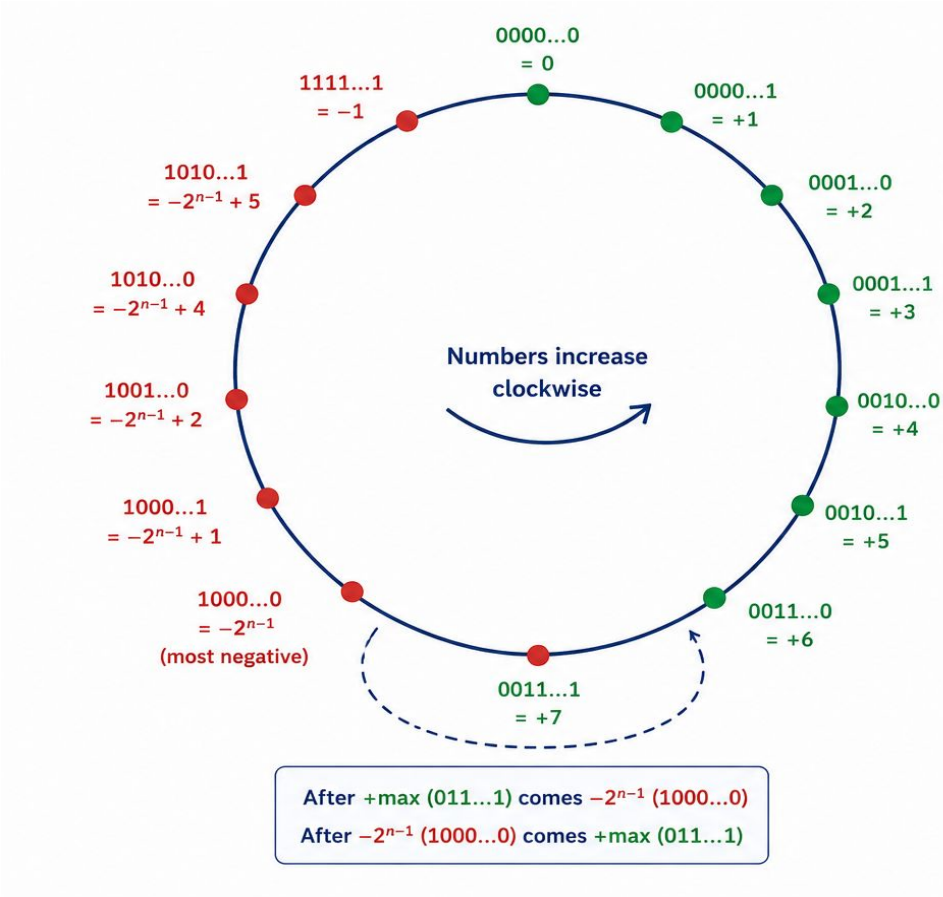
To go from +x to -x OR from -x to +x, the operation is identical: flip all bits, add 1. Symmetric and simple.

QUICK SIGN CHECK

The MSB alone tells you the sign: 0 = non-negative, 1 = negative. No need to decode the whole number to know if it's below zero.

Range & the Number Wheel

For n bits: -2^{n-1} to $+2^{n-1}-1$. And the numbers wrap in a circle.



SIGNED RANGE BY WIDTH

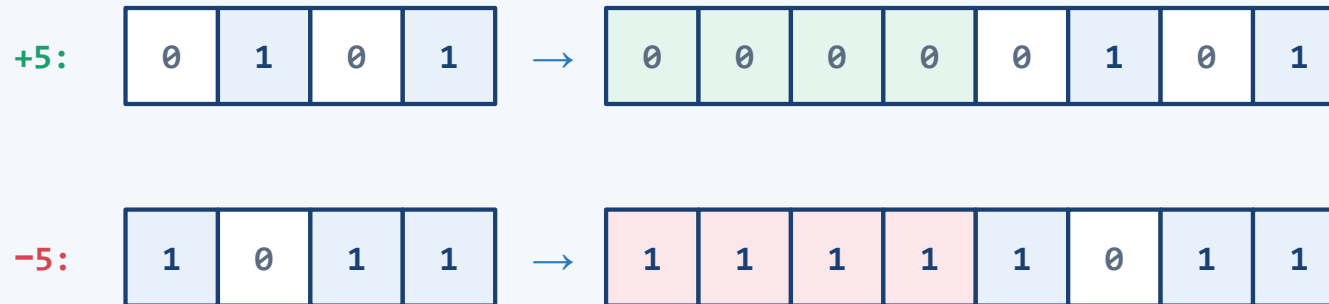
Bits	Min	Max
4	-8	+7
8	-128	+127
16	-32,768	+32,767
32	-2.1 billion	+2.1 billion

Note the asymmetry: one more negative value than positive (because 0 takes a 'positive' slot). That lone extra negative — like -128 in 8 bits — has no positive twin.

Sign Extension

Widening a number? Copy the sign bit — don't just pad with zeros.

To widen 4-bit → 8-bit, repeat the sign bit into the new positions:



The copied sign bits (highlighted) preserve the value. Both still equal ± 5 .

Padding -5 with ZEROS would give 00001011 = +11. Wrong! Always extend the sign.

WHERE YOU HIT THIS

Every time a CPU loads a byte into a 32-bit register.

MIPS & RISC-V have distinct load instructions: 'load byte' (sign-extends) vs 'load byte unsigned' (zero-pads).

Picking the wrong one is a classic bug — you'll meet it again in Module 3.

Quick Check

Show of hands — decode this byte.

Q. In 8-bit two's complement, what decimal value is `10000000` ?

A 0

B -128

C +128

D -0

✓ *MSB weight = -128, rest = 0*

Trap: it looks like -0 (sign bit set, magnitude 0) — but in two's complement there's no -0. It's the most negative value.

3. Binary Arithmetic

Addition works exactly like grade-school carrying. Subtraction is just addition in disguise.

Binary Addition

Four rules, column by column, carrying the 1 — just like decimal.

THE FOUR RULES

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 1 = 10 \quad (0, \text{ carry } 1)$$

$$1 + 1 + 1 = 11 \quad (1, \text{ carry } 1)$$

WORKED: 13 + 11 (4-bit)

carries

A = 13

B = 11

= 24

	1	1	0	1
	1	0	1	1
	1	1	0	0

carry-out (5th bit) means the 4-bit result overflowed — more on that soon.

Subtraction = Adding the Negative

No subtractor circuit needed. Negate the second operand and add.

$$A - B \text{ becomes } A + (-B) = A + (\text{invert } B, \text{ add } 1)$$

1**Take B**

The number you're subtracting.

2**Two's-complement it**

Invert all bits, add 1 → that's -B.

3**Add to A**

Feed A and -B into the SAME adder you already built.

4**Discard final carry**

The carry-out beyond the width is ignored for signed results.

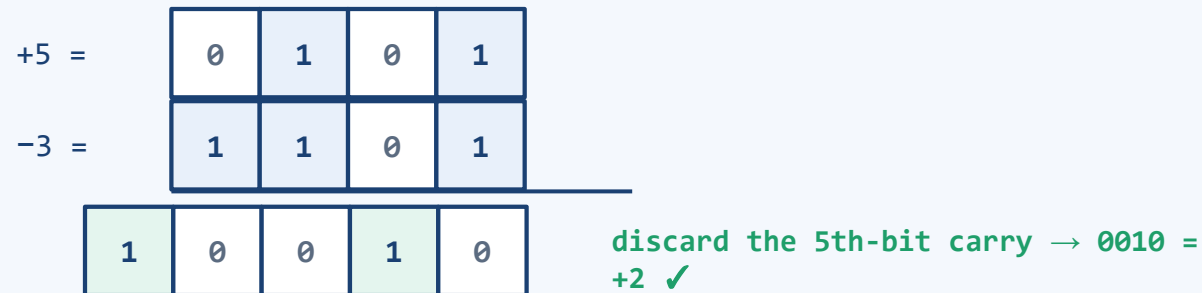
Worked: 5 – 3 in 4-Bit

Watch subtraction happen entirely through addition.

STEP 1 — Encode +3, then negate to get -3



STEP 2 — Add 5 + (-3)



THE MAGIC

5 – 3 = 2. Correct!

We never subtracted. We negated 3 and added.

The carry-out past bit 4 is simply thrown away — and the signed answer is right.

One adder. Add and subtract. This is the entire reason two's complement won.

4. Overflow

When the true answer won't fit in the bits you have — and the result silently lies.

What Is Overflow?

When a correct sum lands outside the representable range, it wraps — silently.

EXAMPLE — 4-bit signed (range -8 to +7)

Add +5 + +4 — both positive, answer should be +9...

+5 =	0	1	0	1
+4 =	0	1	0	0
=	1	0	0	1

= -7 ?!

Two positives gave a NEGATIVE. +9 doesn't fit in 4-bit signed (max +7), so it wrapped past the top of the wheel into negative territory. The hardware didn't crash — it just gave a wrong answer.

THE DANGER

Overflow is SILENT.

No error, no exception by default — just a wrong number that flows on into the next computation.

That's what makes it dangerous. The CPU flags it (next slide) — but only if your code checks the flag.

How Hardware Detects Overflow

One elegant rule: compare the carry INTO and OUT of the sign bit.

THE RULE (signed)

$$\text{Overflow} = \text{Cin}(\text{MSB}) \oplus \text{Cout}(\text{MSB})$$

If the carry INTO the sign-bit column differs from the carry OUT of it, overflow occurred.

Just ONE extra XOR gate on your adder detects every signed overflow. That's the whole cost.

$$\text{Cin}=1, \text{Cout}=0 \rightarrow 1 \oplus 0 = 1 \rightarrow \text{OVERFLOW}$$

THE INTUITIVE VERSION

$$(+)+(+)=(-)$$

Two positives can't sum to a negative → OVERFLOW

$$(-)+(-)=(+)$$

Two negatives can't sum to a positive → OVERFLOW

$$(+)+(-)$$

Opposite signs can NEVER overflow → always safe

Signed vs Unsigned Overflow

Same bits, different overflow rules — the CPU computes both flags every time.

SIGNED overflow

- Detected by: $C_{in} \oplus C_{out}$ at the sign bit.
- Means: the true result fell outside $-2^{n-1} \dots +2^{n-1}-1$.
- Flag name (x86): OF (Overflow Flag).
- Your code reads OF if it cares about signed overflow.

UNSIGNED overflow

- Detected by: the carry-out of the MSB alone.
- Means: the true result exceeded 2^n-1 (didn't fit).
- Flag name (x86): CF (Carry Flag).
- Your code reads CF if the numbers are unsigned.

Key insight: the ALU doesn't know if YOU meant signed or unsigned. It computes both flags; your instructions decide which matters.

Overflow: Four Cases

4-bit signed. Watch which combinations overflow and which don't.

$$+5 + (+4)$$

$$0101 + 0100$$

$$= 1001 = -7$$



OVERFLOW

$$(+) + (+) = (-)$$

$$-6 + (-5)$$

$$1010 + 1011$$

$$= 0101 = +5$$



OVERFLOW

$$(-) + (-) = (+)$$

$$+5 + (-3)$$

$$0101 + 1101$$

$$= 0010 = +2$$



OK

opposite signs

$$+3 + (+2)$$

$$0011 + 0010$$

$$= 0101 = +5$$



OK

fits in range

The pattern is unmistakable: overflow happens ONLY when both inputs share a sign and the result flips it.

When Overflow Broke the Real World

These aren't textbook hypotheticals — they're famous, expensive failures.



Ariane 5 (1996)

A 64-bit float converted to 16-bit signed int overflowed. The rocket self-destructed 37s after launch. Cost: ~\$370 million.



Gangnam Style (2014)

YouTube's view counter was a 32-bit signed int, max ~2.1 billion. The video blew past it — forcing a switch to 64-bit.



Boeing 787 (2015)

A counter overflowing after 248 days of continuous power could shut down all generators. Fix: reboot every 248 days.



Pac-Man level 256

An 8-bit level counter overflowed at 256, corrupting the level into the infamous unplayable 'kill screen'.

Common Mistakes to Avoid

The slips that cost marks — and cause real bugs.



Zero-padding a negative

Widening `-5` as `0000_1011` gives `+11`. Always SIGN-extend, not zero-pad.



Confusing the two overflows

Signed overflow $\neq$ carry-out. Carry-out flags UNSIGNED overflow; sign-bit carry mismatch flags SIGNED.



Forgetting the +1

Two's complement negate is invert AND add 1. One's complement (invert only) is a different, obsolete scheme.



Assuming symmetric range

8-bit is `-128` to `+127`, not `-127` to `+127`. One extra negative, no positive twin.



Misreading the MSB as -0

`10000000` isn't `-0` — there's no `-0` in two's complement. It's the most negative value.



Thinking overflow throws an error

By default it's silent — the wrong number just flows onward. You must check the flag.

Key Takeaways

Five sentences that underpin every integer a computer touches.

- 01** Bits carry no inherent meaning — a number system is a convention you apply (base 2 for machines, base 16 for readable binary).
- 02** One hex digit maps to exactly 4 bits; two hex digits = one byte — that's the entire reason hex exists.
- 03** Two's complement (invert + 1) is the winner: one zero, and one adder handles both addition and subtraction.
- 04** In two's complement the MSB carries a **NEGATIVE** weight, and widening a value means sign-extending, not zero-padding.
- 05** Overflow is silent: it happens when two same-signed numbers produce the wrong sign, detected by $C_{in} \oplus C_{out}$ at the sign bit.

Practice & Homework

Convert, encode, and hunt for overflow.

01

Base Triathlon

Convert 0xB7 to binary and to signed decimal (8-bit). Then -45 to 8-bit two's complement.

02

Two's Complement Lab

Encode +18 and -18 in 8 bits. Verify that negating -18 gives back +18.

03

Overflow Drill

Add several pairs of 4-bit signed numbers; mark which overflow and justify with the sign rule.

EXIT TICKET

1. Convert -45 to 8-bit two's complement.
2. Compute $0110 + 1100$ in 4-bit two's complement — did overflow occur?
3. Convert 0xB7 to binary and to a signed 8-bit decimal value.

Rapid-Fire Recap

Three questions — shout the answers.

Q1

The 8-bit two's complement of +18 is...

11101110

Q2

Signed overflow occurs when the result...

has the wrong sign

Q3

0xFF as an 8-bit SIGNED integer is...

-1

All three instant? You're ready for the labs and L5 (Sequential Circuits — latches & flip-flops).

END OF L04

One adder, every integer.

NEXT — L05

From Combinational to Sequential — latches, flip-flops, and the clock. How circuits gain MEMORY and start to remember what they computed.



**Thanks
for
watching!**